TRƯỜNG ĐẠI HỌC SÀI GÒN

MÔN HỌC: HỌC SÂU

BÀI TẬP LỚN 01

XẤP XỈ HÀM GAUSSIAN 2D BẰNG MẠNG NƠ-RON NHIỀU TẦNG (MLP) TRÊN PYTORCH

Sinh viên thực hiện: Huỳnh Kiện Khải
Mã số sinh viên: 3123411137
Mã nguồn: thư mục mlp_gaussian_approx (có README và lệnh chạy lại)

TP. Hồ Chí Minh, tháng 10 năm 2026

1. Giới thiệu

1.1. Mục tiêu

Báo cáo này khảo sát khả năng xấp xỉ hàm số của mạng nơ-ron nhiều tầng (MLP). Hàm mục tiêu là một hàm Gaussian hai biến. MLP được cài đặt bằng PyTorch và được huấn luyện để dự đoán giá trị hàm từ (x_1, x_2). Ba thí nghiệm được thực hiện:

- **Thí nghiệm A:** thay đổi số mẫu huấn luyện, với mạng 1 tầng ẩn, 3 nơ-ron.
- **Thí nghiệm B:** thay đổi số nơ-ron của một tầng ẩn, từ 1 đến 10.
- **Thí nghiệm C:** thay đổi số tầng ẩn, từ 1 đến 4, mỗi tầng 3 nơ-ron.

1.2. Phát biểu bài toán

Hàm cần xấp xỉ là mật độ Gaussian hai chiều:

$$f(x) = 1 / (2\pi \sqrt{|C|}) \cdot exp(-½ (x - \mu)^T C^{-1} (x - \mu)) \quad (1)$$

với $\mu = [0, 0]^T$ và ma trận hiệp phương sai $C = [[0.3, -0.5], [-0.5, 2]]$. Đầu vào lấy mẫu đều trong miền $x_1 \in [-2, 2]$, $x_2 \in [-4, 4]$. Bài toán là hồi quy: tìm mạng $f_\theta: \mathbb{R}^2 \to \mathbb{R}$ sao cho sai số bình phương trung bình giữa $f_\theta(x)$ và $f(x)$ nhỏ nhất trên dữ liệu chưa thấy.

1.3. Nội dung thực hiện

- Sinh dữ liệu Gaussian, chia train/validation/test với seed cố định và vẽ biểu đồ phân bố.
- Cài đặt MLP, vòng huấn luyện có lưu checkpoint theo validation MSE nhỏ nhất và đánh giá trên tập test bằng PyTorch.
- Chạy ba thí nghiệm A, B, C và so sánh bằng MSE, MAE, RMSE, learning curve và thời gian huấn luyện.
- Chọn mô hình có sai số nhỏ nhất trong ba thí nghiệm A, B, C theo validation MSE, kiểm tra lại và minh họa bề mặt xấp xỉ.
- Tổ chức code của notebook thành các module riêng trong src/ (giữ nguyên hàm, tên biến và cách viết), có requirements.txt, README.md, log CSV, hình và trọng số để chạy lại từ đầu.

2. Dữ liệu

2.1. Công thức và tham số

Nhãn của mỗi mẫu được tính trực tiếp từ công thức (1) với μ và C ở mục 1.2, không thêm nhiễu. Trong hàm Gaussian_2D (src/data_gen.py), với mỗi điểm x ta tính $k = x - \mu$ rồi $y = hệ_số \cdot \exp(-½ k^T C^{-1} k)$, với $hệ_số = 1/(2\pi\sqrt{|C|})$.

2.2. Cách lấy mẫu

Với seed 42, mỗi điểm được lấy mẫu độc lập bằng hàm gen_data_set: $x_1 \sim U(-2, 2)$ và $x_2 \sim U(-4, 4)$. Có hai bộ dữ liệu, giống notebook:

- **Bộ cho mô hình chuẩn:** 1000 mẫu, lưu vào data/DataSet.csv với ba cột x1, x2, y_real.

- **Bộ cho thí nghiệm A, B, C:** 3000 mẫu, chuyển sang float32 và xáo trộn bằng numpy.random.default_rng(42). Thí nghiệm A lấy n mẫu đầu tiên với n thay đổi; thí nghiệm B và C lấy 1000 mẫu đầu tiên.

2.3. Chia dữ liệu

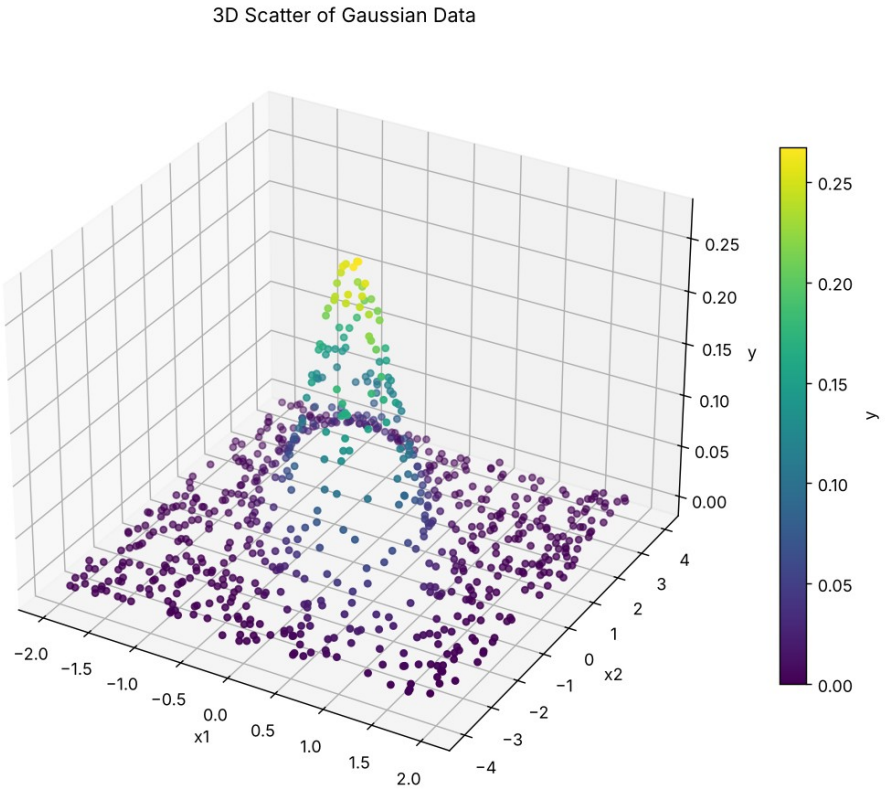
Với mỗi tập n mẫu, hàm data_split_and_to_tensor chia dữ liệu hai lần bằng train_test_split với random_state = 42: 10% làm test, trong 90% còn lại lấy 20% làm validation, phần còn lại là train. Tập validation dùng để chọn checkpoint và chọn kiến trúc; tập test dùng để báo cáo kết quả. Kích thước các tập được cho ở Bảng 1.

Bảng 1. *Kích thước các tập dữ liệu.*

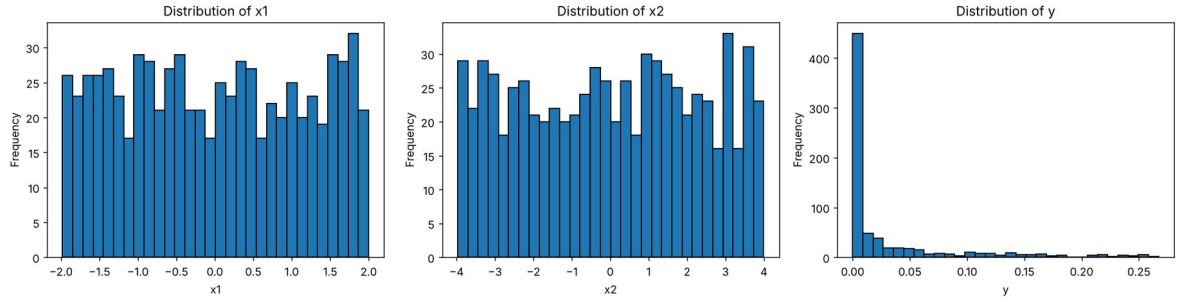
Thí nghiệm	Tổng n	Train	Validation	Test
Mô hình chuẩn, B, C	1000	720	180	100
A	278	200	50	28
A	556	400	100	56
A	834	600	150	84
A	1112	800	200	112
A	1389	1000	250	139
A	1667	1200	300	167
A	1945	1400	350	195
A	2223	1600	400	223
A	2500	1800	450	250
A	2778	2000	500	278

2.4. Biểu đồ dữ liệu

Hình 1 là tập train (720 mẫu) của mô hình chuẩn trong không gian (x_1, x_2, y). Hình 2 là phân bố của x_1, x_2 và y. Hai biến đầu vào phân bố gần đều trong miền lấy mẫu, đúng như kỳ vọng từ phân phối đều. Phân bố của y lệch mạnh về 0: phần lớn mẫu nằm ở vùng "đuôi" phẳng của hàm Gaussian, chỉ một số ít mẫu nằm gần đỉnh.



Hình 1. *Dữ liệu train trong không gian (x1, x2, y); màu thể hiện giá trị y.*



Hình 2. *Phân bố của x1, x2 và y trên tập train.*

3. Mô hình và cài đặt

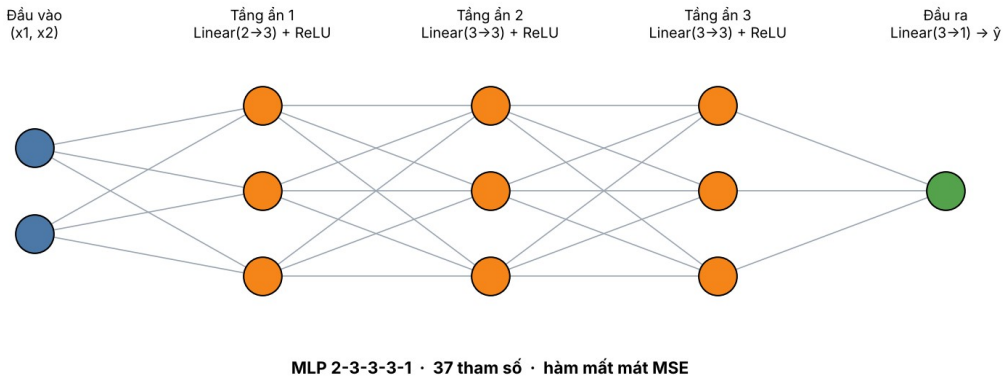
3.1. Kiến trúc MLP

Mạng gồm L tầng ẩn, mỗi tầng là một phép biến đổi tuyến tính theo sau là hàm kích hoạt ReLU, và tầng đầu ra là một phép tuyến tính, không có kích hoạt vì đây là bài toán hồi quy. Với $h^{(0)} = x \in \mathbb{R}^2$, lan truyền thuận (forward) được viết như sau:

$$h^{(l)} = ReLU(W^{(l)} h^{(l-1)} + b^{(l)}),\ l = 1, ..., L \quad (2)$$

$$\hat{y} = W^{(L+1)} h^{(L)} + b^{(L+1)} \quad (3)$$

Hình 3 là sơ đồ kiến trúc 2-3-3-3-1 của mô hình được chọn ở mục 4.6 (3 tầng ẩn, mỗi tầng 3 nơ-ron), kèm kích thước từng lớp và tổng số tham số. Các thí nghiệm dùng chung một lớp MLP: thí nghiệm B đổi số nơ-ron của một tầng ẩn, thí nghiệm C đổi số tầng ẩn.



Hình 3. *Sơ đồ kiến trúc của mô hình được chọn (2-3-3-3-1).*

3.2. Hàm mất mát và các độ đo

Hàm mất mát khi huấn luyện là sai số bình phương trung bình (MSE) trên N mẫu. Khi đánh giá ta báo cáo thêm RMSE và MAE:

$$MSE = (1/N)\,\Sigma\,(\hat{y}_i - y_i)^2,\;\; RMSE = \sqrt{MSE},\;\; MAE = (1/N)\,\Sigma\,|\hat{y}_i - y_i| \qquad (4)$$

3.3. Thiết lập huấn luyện

Mọi thí nghiệm dùng chung một thiết lập (Bảng 2). Trước mỗi lần huấn luyện, ta gọi torch.manual_seed(42) để khởi tạo trọng số và thứ tự xáo trộn batch giống nhau giữa các lần chạy lại.

Bảng 2. *Siêu tham số và thiết lập chung.*

Thành phần	Giá trị
Seed	42
Bộ tối ưu	Adam, learning rate = 0.002
Hàm kích hoạt	ReLU (tầng ẩn); tầng ra tuyến tính
Hàm mất mát	MSE
Số epoch	1000
Batch size	32 (mô hình chuẩn dùng full-batch 720 mẫu)
Chọn checkpoint	Epoch có validation MSE nhỏ nhất
Thiết bị	CPU
Thí nghiệm A	1 tầng ẩn × 3 nơ-ron; số mẫu train 200 ... 2000
Thí nghiệm B	n = 1000; 1 tầng ẩn, B = 1 ... 10 nơ-ron
Thí nghiệm C	n = 1000; C = 1 ... 4 tầng ẩn, mỗi tầng 3 nơ-ron

3.4. Các đoạn mã quan trọng

Code trong src/ được lấy từ notebook của bài (giữ nguyên tên hàm, tên biến và cách viết), chỉ đổi phần lưu và đọc file. Lớp MLP (file src/model.py) xây mạng từ danh sách số nơ-ron của các tầng ẩn hiden_sizes. Mỗi phần tử của hiden_sizes tạo thêm một cặp Linear và ReLU; cuối cùng thêm tầng Linear đầu ra.

```
class MLP(nn.Module):
  def __init__(self, input_size = 2, hiden_sizes = [3], output_size = 1):
    super().__init__()
    layers = []
    prev_input = input_size

    for hiden_size in hiden_sizes:# tạo ra len(hiden_sizes) hiden layers
      layers.append(nn.Linear(prev_input, hiden_size))
      layers.append(nn.ReLU()) # hàm kích hoạt ReLU
      prev_input = hiden_size

    layers.append(nn.Linear(prev_input, output_size))
    self.network = nn.Sequential(*layers)

  def forward(self, x):
      return self.network(x)
```

Hàm Train (file src/train.py) gồm hai pha ở mỗi epoch. Ở pha train, với mỗi batch ta xóa gradient, tính mất mát, lan truyền ngược và cập nhật trọng số. Ở pha validation, mô hình chuyển sang eval() và tính MSE trên toàn bộ tập validation trong torch.no_grad(), tức là validation không cập nhật trọng số. Checkpoint chỉ được ghi khi validation MSE tốt hơn mọi epoch trước.

```
    for X_batch, y_batch in train_loader:
      X_batch, y_batch = X_batch.to(device), y_batch.to(device)
      optimizer.zero_grad() # reset grad về 0 để không bị cộng dồn
      y_pred = model(X_batch).squeeze(-1) # train + predict
      loss = criterion(y_pred, y_batch) # tính loss
      loss.backward() # backpropagation
      optimizer.step()
      running_loss += loss.item() * X_batch.size(0) # cộng dồn loss

    # validation
    model.eval()
    with torch.no_grad():
      y_val_pred = model(X_val_tensor).squeeze(-1)
      val_loss = criterion(y_val_pred, y_val_tensor)

    # lưu checkpoint min_vali_loss
    if val_loss.item() < min_vali_loss:
      min_vali_loss = val_loss.item()
      best_epoch = epoch + 1
      torch.save({"epoch": epoch + 1, "model_state_dict": model.state_dict(),
                  "optimizer_state_dict": optimizer.state_dict(),
                  "val_loss": min_vali_loss}, file)
```

Sau khi huấn luyện, hàm evaluate (file src/evaluate.py) nạp lại checkpoint tốt nhất và tính MSE, RMSE, MAE trên tập test. Như vậy số liệu test ứng với epoch được chọn bằng validation, không phải epoch cuối.

```
def evaluate(model, file, X_test, y_test):
    checkpoint = torch.load(file, map_location=device, weights_only=True)
    model.load_state_dict(checkpoint["model_state_dict"])
    model.eval()
    with torch.no_grad():
        pred = model(X_test.to(device)).squeeze(-1).cpu()

    err = pred - y_test
    mse = (err ** 2).mean().item()
    return {"test_mse": mse, "test_rmse": mse ** 0.5, "test_mae":
err.abs().mean().item()}
```

3.5. Môi trường chạy

Python 3.13, PyTorch 2.14 (chạy trên CPU), NumPy 2.5, pandas 3.0, scikit-learn 1.9, Matplotlib 3.11. Danh sách phụ thuộc nằm trong requirements.txt; các đường dẫn trong mã đều tương đối so với thư mục gốc dự án.

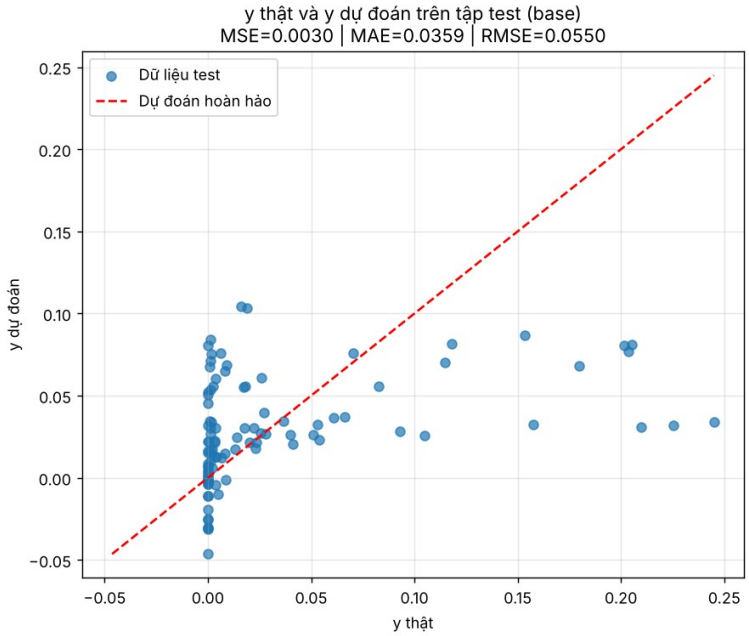
4. Thực nghiệm

4.1. Quy trình đánh giá

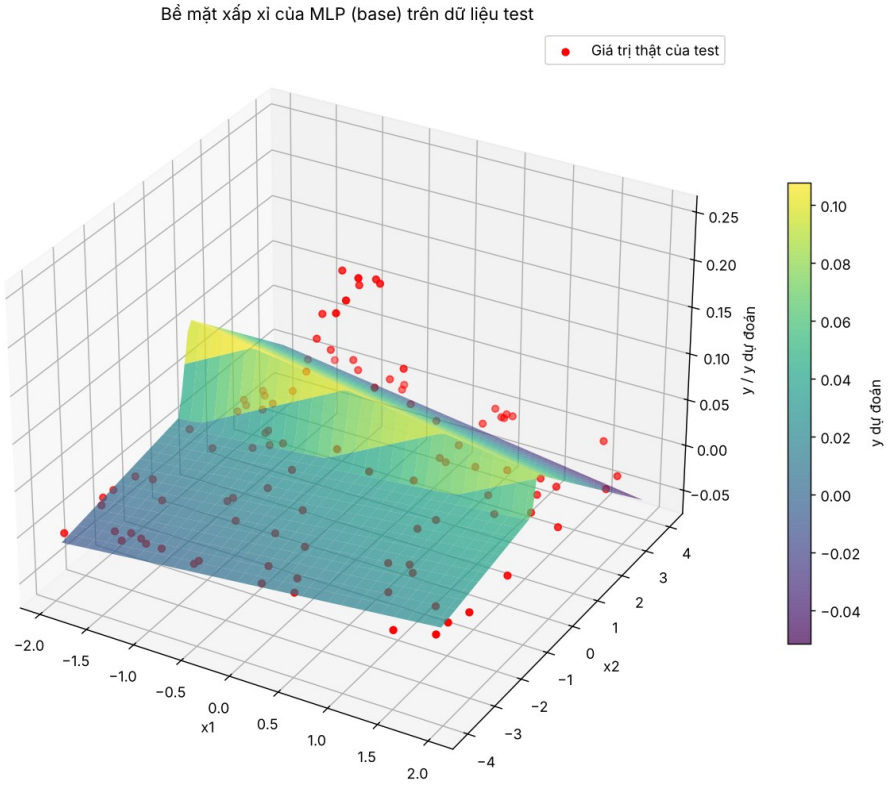
Các cấu hình trong cùng một thí nghiệm chỉ khác nhau đúng yếu tố đang khảo sát: cùng seed, bộ tối ưu, learning rate, batch size, số epoch và cách chia dữ liệu. Checkpoint của mỗi cấu hình được chọn theo validation MSE, và test được tính cho checkpoint đó. Với B và C, mọi cấu hình dùng cùng 1000 mẫu và cùng tập test 100 mẫu. Với A, mỗi n được chia lại theo cùng tỉ lệ, nên kích thước tập test thay đổi theo n (từ 28 đến 278 mẫu, Bảng 1); điều này cần lưu ý khi đọc kết quả A.

4.2. Mô hình chuẩn

Mô hình chuẩn là mạng 1 tầng ẩn × 3 nơ-ron, huấn luyện full-batch trên 720 mẫu. Kết quả trên test: MSE = 3.03×10^{-3}, RMSE = 0.0550, MAE = 0.0359; epoch tốt nhất là 1000. Hình 4 so sánh y dự đoán với y thật trên tập test và Hình 5 là bề mặt xấp xỉ của mô hình cùng các điểm test. Mạng 3 nơ-ron chưa bắt được đỉnh của hàm: các mẫu có y lớn bị dự đoán thấp hơn nhiều (Hình 4) và bề mặt chỉ gồm vài mảnh phẳng (Hình 5).



Hình 4. *y thật và y dự đoán trên tập test của mô hình chuẩn.*



Hình 5. *Bề mặt xấp xỉ của mô hình chuẩn và các điểm test.*

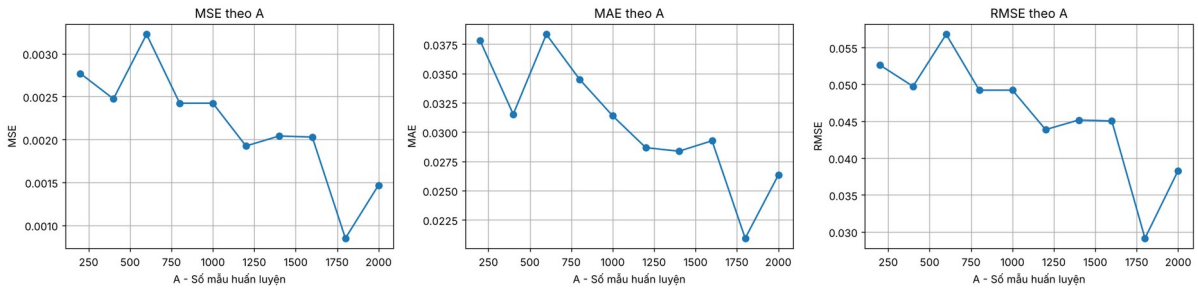
4.3. Thí nghiệm A: số mẫu huấn luyện

Mạng 1 tầng ẩn × 3 nơ-ron được huấn luyện với số mẫu train từ 200 đến 2000 (Bảng 3, Hình 6).

Bảng 3. *Kết quả thí nghiệm A (đo trên test).*

A (số mẫu train)	MSE	MAE	RMSE
200	2.77×10^{-3}	0.0378	0.0526

A (số mẫu train)	MSE	MAE	RMSE
400	2.47×10^{-3}	0.0315	0.0497
600	3.23×10^{-3}	0.0384	0.0568
800	2.42×10^{-3}	0.0345	0.0492
1000	2.42×10^{-3}	0.0314	0.0492
1200	1.92×10^{-3}	0.0287	0.0439
1400	2.04×10^{-3}	0.0284	0.0452
1600	2.03×10^{-3}	0.0293	0.0450
1800	8.48×10^{-4}	0.0209	0.0291
2000	1.47×10^{-3}	0.0263	0.0383



Hình 6. *MSE, MAE và RMSE trên test theo số mẫu train A.*

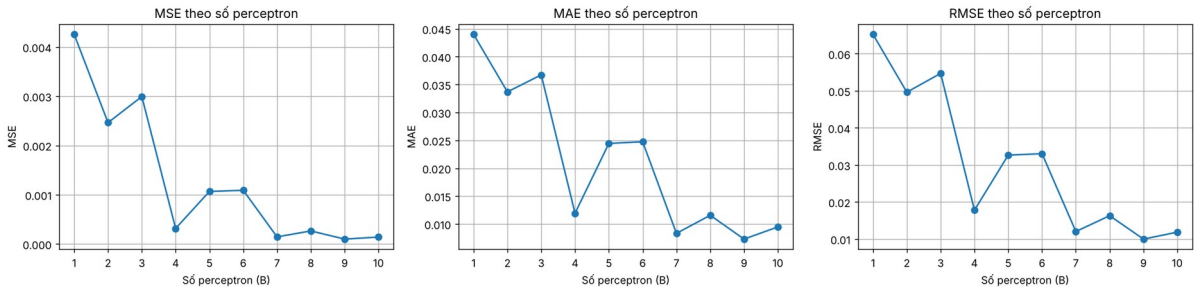
MSE test giảm từ 2.77×10^{-3} (A = 200) xuống 1.47×10^{-3} (A = 2000) và thấp nhất là 8.48×10^{-4} tại A = 1800. Xu hướng chung là giảm, nhưng đường cong không đơn điệu: A = 600 có MSE cao nhất (3.23×10^{-3}), và A = 2000 cao hơn A = 1800. MAE và RMSE có cùng xu hướng với MSE.

4.4. Thí nghiệm B: số nơ-ron của tầng ẩn

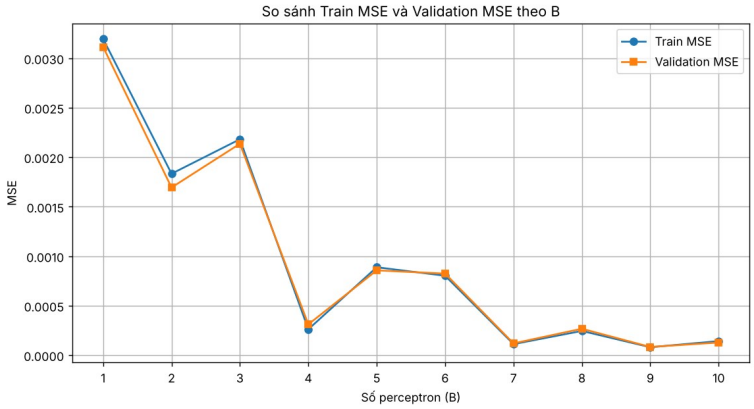
Với 1000 mẫu và 1 tầng ẩn, số nơ-ron B tăng từ 1 đến 10 (Bảng 4, Hình 7, 8, 9).

Bảng 4. *Kết quả thí nghiệm B (1 tầng ẩn, n = 1000).*

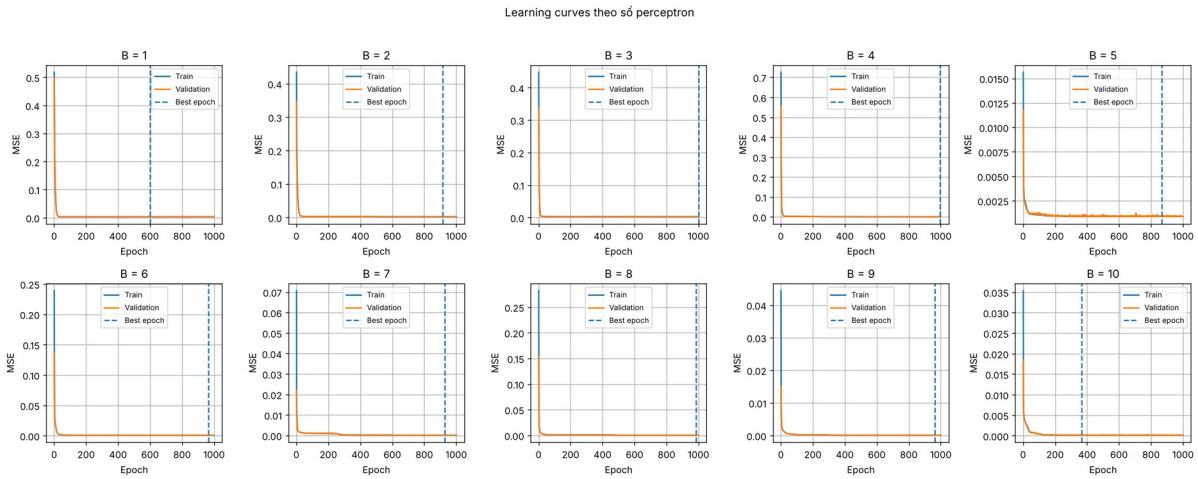
B	Epoch tốt nhất	Train MSE	Val MSE	Test MSE	Test MAE	Test RMSE
1	599	3.20×10^{-3}	3.11×10^{-3}	4.25×10^{-3}	0.0440	0.0652
2	914	1.83×10^{-3}	1.69×10^{-3}	2.46×10^{-3}	0.0337	0.0496
3	999	2.18×10^{-3}	2.14×10^{-3}	2.99×10^{-3}	0.0367	0.0547
4	995	2.61×10^{-4}	3.11×10^{-4}	3.16×10^{-4}	0.0119	0.0178
5	866	8.88×10^{-4}	8.57×10^{-4}	1.07×10^{-3}	0.0244	0.0327
6	966	8.02×10^{-4}	8.24×10^{-4}	1.09×10^{-3}	0.0247	0.0330
7	929	1.12×10^{-4}	1.19×10^{-4}	1.45×10^{-4}	0.0083	0.0120
8	983	2.45×10^{-4}	2.67×10^{-4}	2.65×10^{-4}	0.0116	0.0163
9	962	8.01×10^{-5}	8.30×10^{-5}	9.95×10^{-5}	0.0073	0.0100
10	367	1.42×10^{-4}	1.28×10^{-4}	1.40×10^{-4}	0.0095	0.0118



Hình 7. *MSE, MAE và RMSE trên test theo số nơ-ron B.*



Hình 8. *Train MSE và Validation MSE theo số nơ-ron B.*



Hình 9. *Learning curve (train, validation, epoch tốt nhất) của các mô hình B = 1 ... 10.*

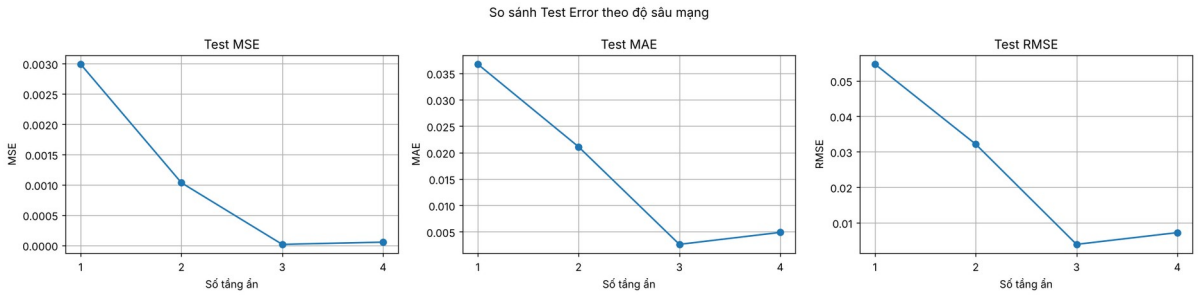
B = 1 cho kết quả kém nhất (MSE test 4.25×10^{-3}). Khi tăng B, sai số nhìn chung giảm và tốt nhất là B = 9 với 9.95×10^{-5}. Train MSE, Validation MSE và Test MSE cùng bậc và chênh lệch nhỏ ở mọi B, nên chưa thấy dấu hiệu overfitting. Hình 9 cho thấy train và validation giảm nhanh trong vài chục epoch đầu rồi giảm chậm dần; với phần lớn cấu hình, epoch tốt nhất nằm gần 1000 (Bảng 4).

4.5. Thí nghiệm C: số tầng ẩn

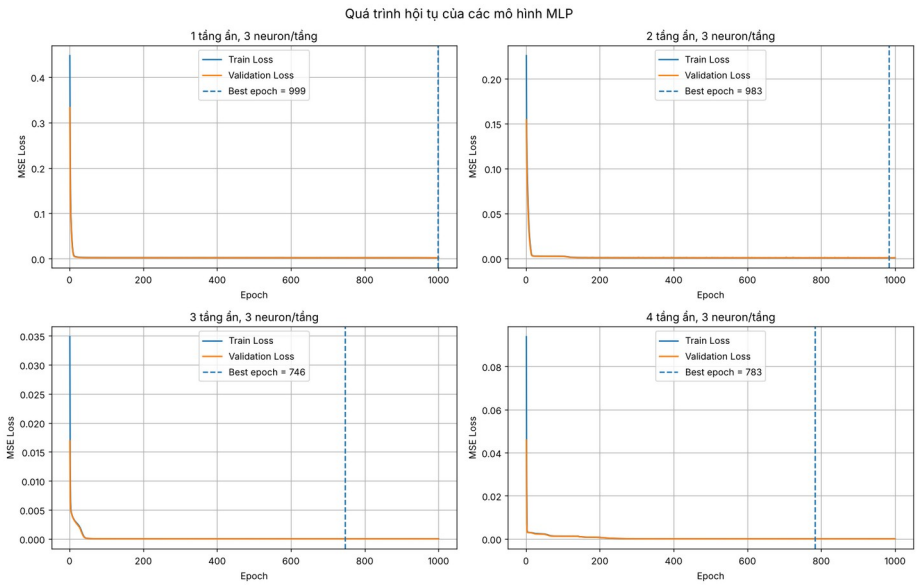
Với 1000 mẫu, số tầng ẩn C tăng từ 1 đến 4, mỗi tầng 3 nơ-ron (Bảng 5, Hình 10, 11, 12).

Bảng 5. *Kết quả thí nghiệm C (mỗi tầng ẩn 3 nơ-ron, n = 1000).*

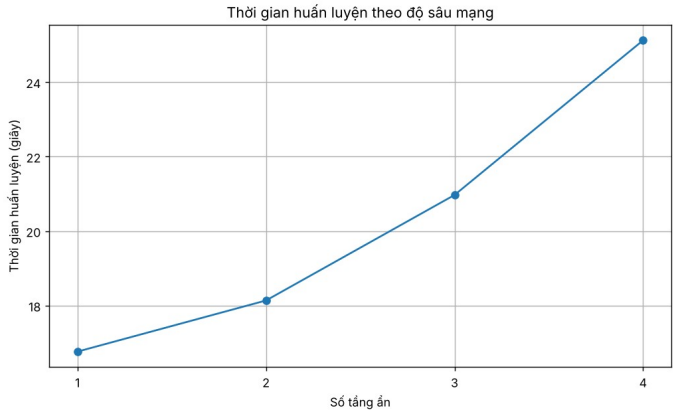
Số tầng ẩn	Epoch tốt nhất	Train MSE	Val MSE	Test MSE	Test MAE	Test RMSE	Thời gian (s)
1	999	2.18×10^{-3}	2.14×10^{-3}	2.99×10^{-3}	0.0367	0.0547	16.76
2	983	7.56×10^{-4}	7.86×10^{-4}	1.03×10^{-3}	0.0211	0.0321	18.14
3	746	1.23×10^{-5}	1.22×10^{-5}	1.52×10^{-5}	0.0026	0.0039	20.97
4	783	4.85×10^{-5}	5.04×10^{-5}	5.20×10^{-5}	0.0049	0.0072	25.12



Hình 10. *MSE, MAE và RMSE trên test theo số tầng ẩn.*



Hình 11. *Learning curve của các mô hình 1 ... 4 tầng ẩn.*



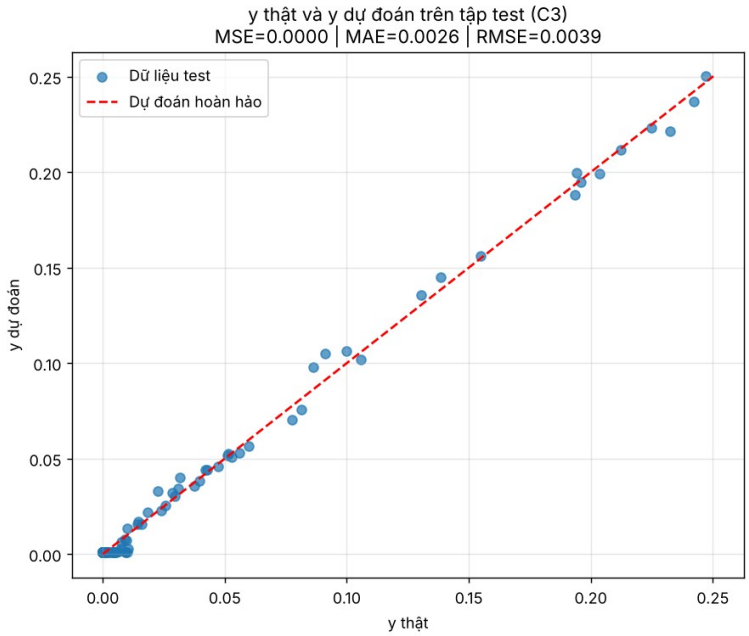
Hình 12. *Thời gian huấn luyện theo số tầng ẩn.*

Sai số giảm rất nhanh khi thêm tầng ẩn: MSE test là 2.99×10^{-3} (1 tầng), 1.03×10^{-3} (2 tầng) và 1.52×10^{-5} (3 tầng), thấp hơn 1 tầng khoảng 197 lần. Mạng 4 tầng cho 5.20×10^{-5}, kém hơn mạng 3 tầng. Thời gian huấn luyện tăng theo độ sâu, từ 16.76 s lên 25.12 s (Hình 12).

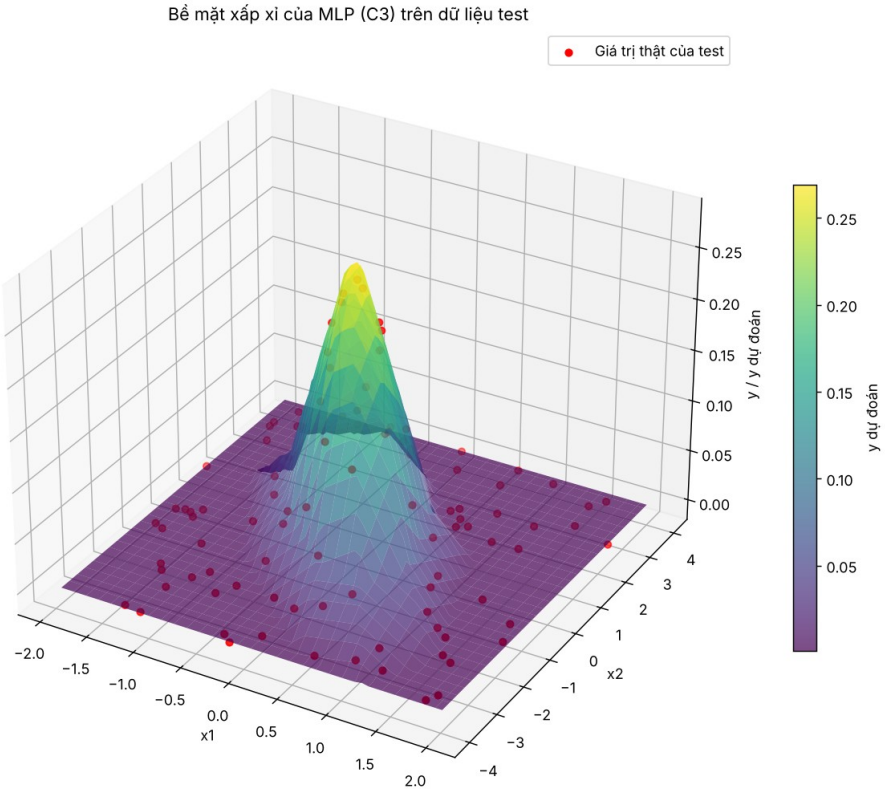
4.6. Kiến trúc được chọn và minh họa

Trong tất cả cấu hình của ba thí nghiệm A, B, C, mô hình có validation MSE nhỏ nhất (và cũng có test MSE nhỏ nhất) là **3 tầng ẩn, mỗi tầng 3 nơ-ron (2-3-3-3-1)**, với Val MSE = 1.22×10^{-5}. Trên tập test (100 mẫu), mô hình đạt MSE = 1.52×10^{-5}, RMSE = 0.0039 và MAE = 0.0026. Trọng số được lưu tại weights/selected_model.pt. Hình 13 so sánh y dự đoán với y thật trên tập test, và Hình 14 là bề mặt xấp xỉ của mô hình cùng các điểm test.

Mô hình được kiểm tra lại bằng src/select_model.py: nạp weights/selected_model.pt cho đúng Val MSE và Test MSE đã lưu; tập train, validation và test (720/180/100) không giao nhau; cấu trúc lớp và số tham số (37) khớp với cấu hình. Kết quả kiểm tra ghi trong results/selection_check.json.



Hình 13. *y thật và y dự đoán trên tập test của kiến trúc được chọn.*



Hình 14. *Bề mặt xấp xỉ của kiến trúc được chọn và các điểm test.*

5. Thảo luận

5.1. Diễn giải xu hướng

- **Số mẫu (A):** thêm dữ liệu giúp giảm sai số, nhưng mức giảm khiêm tốn (MSE vẫn ở cỡ 10^{-3}) so với việc đổi kiến trúc ở B và C. Với mạng chỉ gồm 1 tầng ẩn × 3 nơ-ron, sai số có vẻ bị giới hạn bởi cấu trúc mạng hơn là bởi lượng dữ liệu.
- **Số nơ-ron (B):** nhiều nơ-ron hơn cho sai số thấp hơn nhìn chung; từ B = 1 đến B = 9 sai số giảm hơn 40 lần.
- **Số tầng ẩn (C):** thêm tầng ẩn giảm sai số mạnh nhất: 3 tầng thấp hơn 1 tầng khoảng 200 lần, trong khi thời gian huấn luyện chỉ tăng khoảng 1.5 lần từ 1 lên 4 tầng.

5.2. Các trường hợp không như kỳ vọng

- **A = 600 tệ hơn A = 400, và A = 2000 tệ hơn A = 1800.** Ở thí nghiệm A, tập test thay đổi theo n (28 mẫu ở A = 200, 84 mẫu ở A = 600, 278 mẫu ở A = 2000), nên mỗi điểm được đo trên một tập khác nhau; với tập test nhỏ, sai số đo được nhiễu hơn. Mỗi cấu hình cũng chỉ chạy một seed.
- **B = 3 kém hơn B = 2, B = 4 tốt hơn nhiều so với B = 5 và 6, B = 10 kém hơn B = 9.** Mỗi cấu hình chỉ chạy một seed và phần lớn có epoch tốt nhất gần 1000 (Bảng 4), tức là chưa chắc đã hội tụ. Vì vậy chênh lệch giữa các B lân cận có thể đến từ lần khởi tạo và độ dài huấn luyện chứ không chỉ từ số nơ-ron.
- **Mạng 4 tầng kém hơn mạng 3 tầng.** Mỗi cấu hình chỉ chạy một seed (Hình 11, Bảng 5), nên chưa đủ để kết luận mạng 4 tầng luôn kém hơn mạng 3 tầng.

5.3. Hạn chế

- Mỗi cấu hình chỉ chạy một seed, chưa có trung bình và độ lệch chuẩn nên khó kết luận chắc chắn với các chênh lệch nhỏ.
- Nhiều mô hình có epoch tốt nhất gần 1000 (Bảng 4, 5), nên có thể chưa hội tụ hoàn toàn.
- Ở thí nghiệm A, tập test khác nhau giữa các n nên các điểm không so sánh trực tiếp hoàn toàn.
- Đầu vào và đầu ra không được chuẩn hóa.
- Thời gian huấn luyện chỉ đo một lần trên CPU, có thể chịu ảnh hưởng của tải máy.

5.4. Đề xuất cải tiến

- Chạy mỗi cấu hình với nhiều seed và báo cáo trung bình ± độ lệch chuẩn.
- Dùng tập test cố định cho thí nghiệm A để các điểm so sánh trực tiếp được.
- Dùng bộ lập lịch learning rate hoặc huấn luyện lâu hơn để các mô hình hội tụ đầy đủ.
- Chuẩn hóa đầu vào và đầu ra.

6. Kết luận

Báo cáo đã cài đặt MLP bằng PyTorch để xấp xỉ hàm Gaussian 2D và khảo sát ba yếu tố. Kết quả chính: (1) tăng số mẫu từ 200 lên 2000 với mạng 1 tầng ẩn × 3 nơ-ron giảm MSE test từ 2.77×10^{-3} xuống 1.47×10^{-3}; (2) tăng số nơ-ron của một tầng ẩn giảm sai số rõ rệt, tốt nhất ở B = 9 (9.95×10^{-5}); (3) tăng số tầng ẩn hiệu quả nhất: mạng 3 tầng × 3 nơ-ron đạt MSE test 1.52×10^{-5}. Các điểm không đơn điệu giữa những cấu hình lân cận có thể do tập test thay đổi (thí nghiệm A), do chỉ chạy một seed và do quá trình huấn luyện. Hướng tiếp theo là chạy nhiều seed, dùng tập test cố định cho A và huấn luyện đầy đủ hơn.

Tài liệu tham khảo

[1] I. Goodfellow, Y. Bengio, A. Courville. Deep Learning. MIT Press, 2016.

[2] G. Cybenko. Approximation by superpositions of a sigmoidal function. Mathematics of Control, Signals and Systems, 2(4), 303–314, 1989.

[3] K. Hornik, M. Stinchcombe, H. White. Multilayer feedforward networks are universal approximators. Neural Networks, 2(5), 359–366, 1989.

[4] D. P. Kingma, J. Ba. Adam: A method for stochastic optimization. ICLR, 2015.

[5] A. Paszke et al. PyTorch: An imperative style, high-performance deep learning library. NeurIPS, 2019.

Phụ lục: lệnh tái lập

Chạy từ thư mục gốc dự án mlp_gaussian_approx/ sau khi cài pip install -r requirements.txt:

```
python -m src.run_experiments --exp all   # mô hình chuẩn + A + B + C
python -m src.select_model                # chọn mô hình tốt nhất trong A, B, C
và kiểm tra
python -m src.plots                       # xuất hình và bảng
python -m src.diagram                     # sơ đồ kiến trúc (Hình 3)
node src/build_report.js                  # dựng báo cáo .docx (cần: npm i docx)
```

Chạy riêng từng thí nghiệm: --exp base | A | B | C; chạy thử nhanh: thêm --epochs 20. Cấu trúc thư mục: src/ (mã nguồn), data/ (DataSet.csv), logs/ (history_*.csv và log chạy), results/ (bảng kết quả CSV), figures/ (hình), weights/ (trọng số .pt; selected_model.pt là kiến trúc được chọn), docs/ (báo cáo, slide, notebook gốc, LOG.md).